Pokémon Strength Classifier

Play

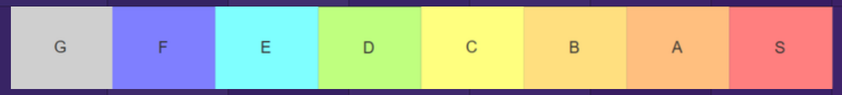
Exit

Problem & Dataset

Big Question:

Can machine learning predict Pokémon strength tiers using only combat statistics?

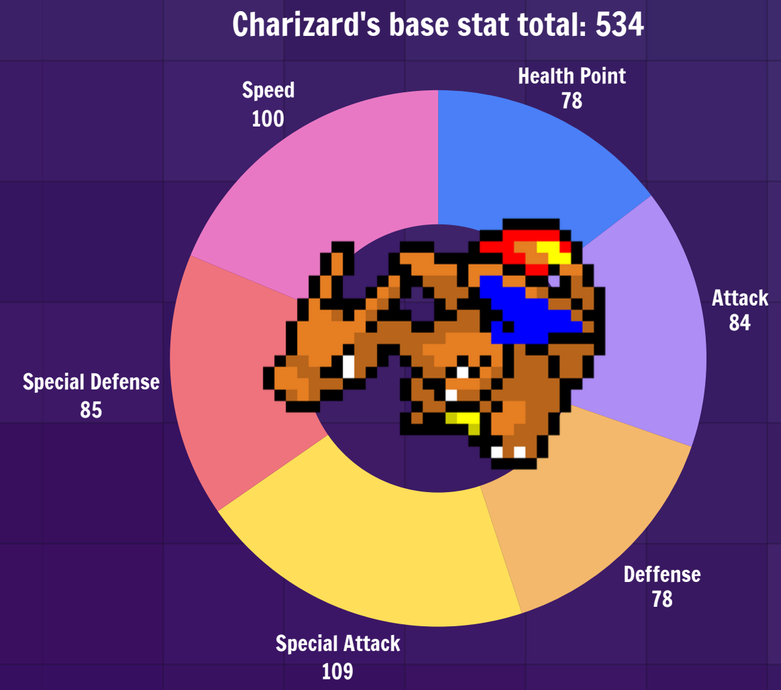
- Every Pokémon are ranked into 8 tiers



- Goal: build a machine learning model that classifies Pokemon into 8 power tiers (G through S) based on their combat statistics.

Dataset

- Dataset size: 1350 Samples
- Input Features

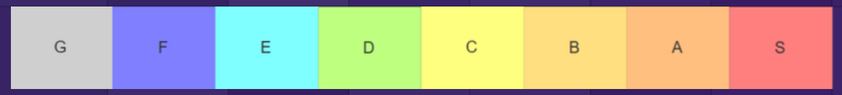


Problem & Dataset

Big Question:

Can machine learning predict Pokémon strength tiers using only combat statistics?

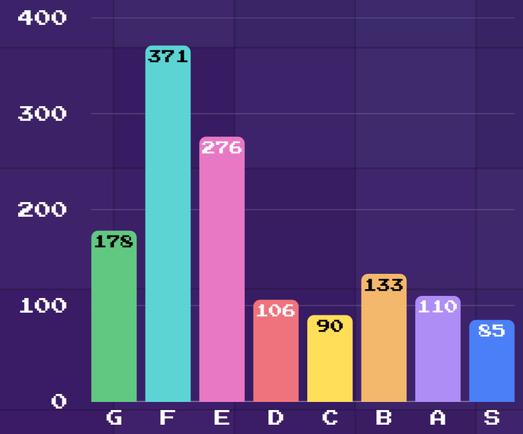
- Every Pokémon are ranked into 8 tiers



- Goal: build a machine learning model that classifies Pokemon into 8 power tiers (G through S) based on their combat statistics.

Dataset

- Dataset size: 1350 Samples
- Input Features
HP, Atk, Def, SpeA, SpeD, Sp, Base stat total
- Target Output:
Tier Label (G → S)



Data Preprocessing

Data cleaning
- Missing values handled
- Labels standardized

→

Train/Dev/Test split
- Used for fair model evaluation

→

Dimension Reduction
- Applied using `Principal Component Analysis`

ML Approach & Baseline Model

Pipeline

Data cleaning: Clean the Power Ranking column, map G-S → 0-7

↓

Load data

↓

Feature selection: 7 combat stats as X, Power Ranking as y

↓

Data splitting: 70% train / 15% dev / 15% test

↓

Data preprocessing
StandardScaler and PCA

↓

Train Model: MLPClassifier neural network

↓

Evaluation and Improve

Baseline Model

- Model configuration

Parameter	Value	Purpose
hidden_layer_sizes	(10,)	1 hidden layer, 10 neurons
activation	relu	Passes positive values unchanged, zeros negatives
alpha	0.0001	L2 regularization — penalizes large weights
max_iter	1000	Max optimization iterations
random_state	42	Fixed seed for reproducibility

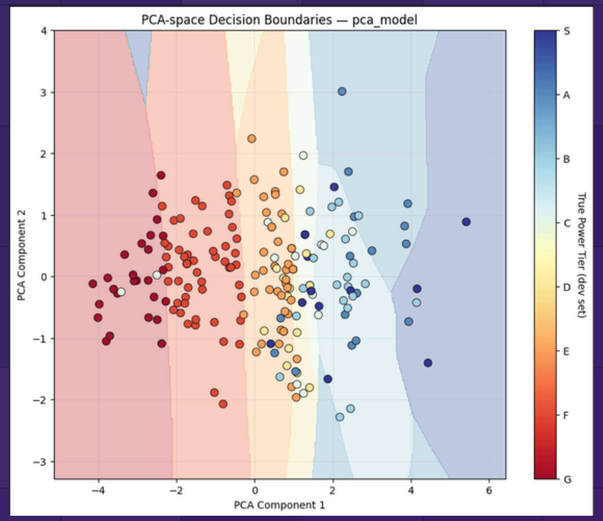
We use the MLP (Multi-Layer Perceptron) from Scikit-learn because:

* It can learn nonlinear relationships.
* Pokémon statistics may interact in complex ways.
* It provides a strong baseline for classification problems.

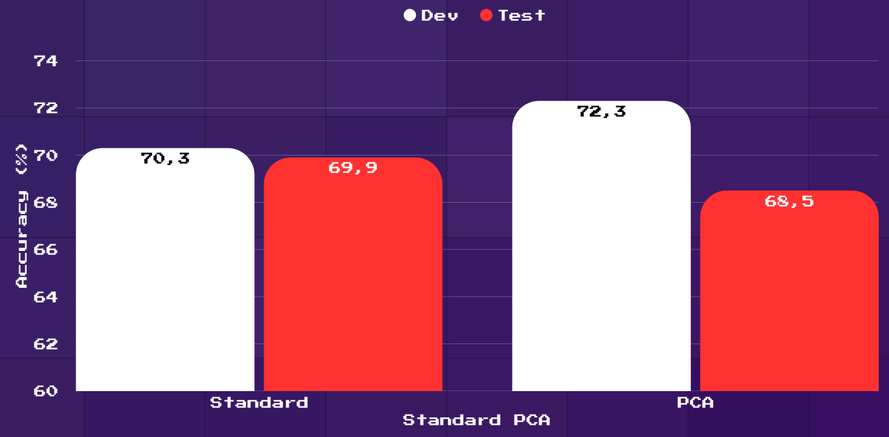
RESULTS & METRICS

Result

Decision Boundaries



Model Performance Comparison



Comparison Table

Model	Train Acc	Dev Acc	Gap	Status
Standard — alpha=-1, iter=3, h=10 (ideal)	0.744	0.703	0.041	Good fit ★
Standard — h=50 (best dev)	0.807	0.708	0.099	Overfit
Standard — h=100 (large)	0.818	0.698	0.120	Overfit
PCA — h=(100,1), alpha=-4, iter=500	0.739	0.7227	0.017	Good fit ★

★ Both the ideal standard model and the best PCA model maintain a tight train–dev gap. The PCA model edges ahead on dev accuracy while also being less prone to overfitting at higher capacities — making it the stronger candidate for further development.

- **Key Finding:**

PCA improved development accuracy, but the Standard model generalized more consistently on unseen data

- **WHY THIS MATTERS:**

Higher dev accuracy does not always mean better performance on unseen data.

MODEL ERRORS

- Prediction

Actual: S
Predicted: E
Error: -5 tiers

- Sneasler represents an outlier case where numerical features fail to represent true performance

- Prediction

Actual: S
Predicted: A
Error: -1 tier

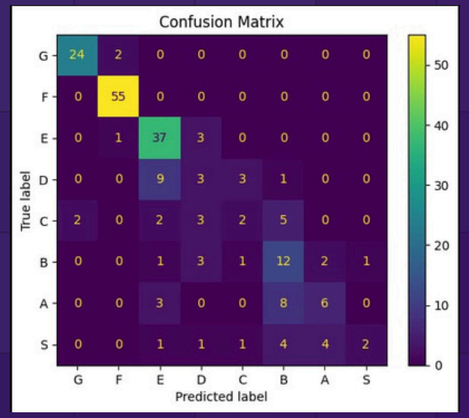
- The model understood that Annihilape was strong, but underestimated its true tier

- Prediction

Actual: A
Predicted: E
Error: -3 tiers

- Feature overlap caused confusion between distant tiers

Confusion Matrix



What the Matrix Shows

Pattern 1
Adjacent-tier confusion

Most mistakes occurred between neighboring tiers
(B ↔ C, C ↔ D)

Pattern 2
Mid-tier problem

D and C tiers had the lowest accuracy
→ Overlapping stat profiles
+ limited samples

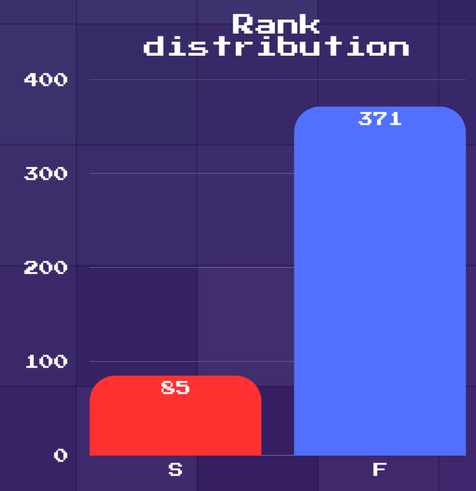
Pattern 3
Rare outliers

Some Pokémon were severely underestimated (e.g., Pikachu, Corviknight)
→ Missing contextual features

=> | Stats alone cannot explain strength

WHY DID ERRORS HAPPEN?

Class Imbalance



=> The model learned majority classes more effectively than minority tiers

Missing Context

- Model only used:

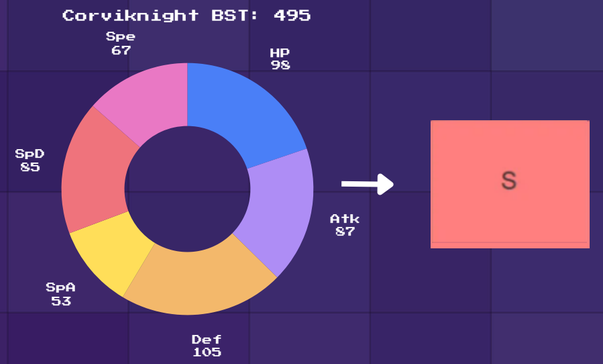
✓ HP
✓ Attack
✓ Defense
✓ Speed
✓ BST

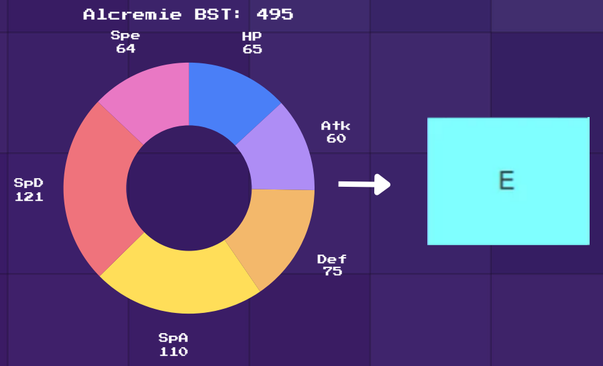
- Missing:

✗ Pokémon Type
✗ Abilities
✗ Legendary status
✗ Items hold

=> Important contextual information was not available to the model

Statistical Overlap





=> Similar statistics did not always imply similar strength tiers

Key Insight

Most prediction errors were caused by dataset limitations rather than model failure alone

AI-ASSISTED IMPROVEMENT

AI Suggestion

"How can we improve our Pokémon classifier?"

AI recommendations

- ✔ Increase hidden neurons
- ✔ Try PCA
- ✔ Add richer features
- ✔ Tune hyperparameters

- **Tested:**

1. Apply PCA to Reduce Collinearity
2. Increase hidden neurons

EVIDENCE

- **Evidence A — Hidden Neuron Experiment**

A grid search was conducted over alpha_power, max_iter_power, and hidden_layer_amt, where alpha = $10^{\text{alpha\_power}}$ and max_iter = $10^{\text{max\_iter\_power}}$. Dev accuracy is the primary selection criterion.

alpha_power	max_iter_power	hidden	Fit?	Train Acc	Dev Acc	F1
1	3	10	underfit	0.650	0.639	0.56
1	4	10	underfit	0.650	0.639	0.56
-1	3	20	overfit	0.784	0.678	0.65
-1	3	50	overfit	0.807	0.708	0.69
-1	3	100	overfit	0.818	0.698	0.68

★ Selected as ideal: alpha=0.1, max_iter=1000, hidden=10 → dev accuracy 70.3%

=> Larger networks slightly improved accuracy but increased overfitting

- **Evidence B — PCA Improvement**

Criterion	Standard Model	PCA Model (best)
Input dimensions	7 features	2 PCA components
Best dev accuracy	0.703	**0.7227**
Train accuracy	0.744	0.739
Train–dev gap	0.041	**0.017**
Fit assessment	Good fit	Good fit
F tier recall	—	1.00
E tier recall	—	0.90
Mid-tier (D/C) perf.	Poor	Still poor

- PCA achieves a marginally better dev accuracy (0.7227 vs 0.703) with a tighter train–dev gap.
- The top-volume tiers (F, E) improve notably under PCA, suggesting the principal components capture a cleaner separation for high-population classes.
- Mid-tiers (D, C) remain problematic under both models — insufficient samples and overlapping stat ranges are the limiting factor, not the dimensionality.

==> AI generated hypotheses, not guaranteed improvements

Did the AI assistant help or mislead us?

AI HELPED

1. Generated improvement ideas

Suggested:
- PCA
- Hidden neurons
- Hyperparameter tuning

2. Encouraged experimentation

Helped us test
multiple model variants

3. Improved interpretation

Helped explain
overfitting and errors

Criterion	Standard Model	PCA Model (best)
Input dimensions	7 features	2 PCA components
Best dev accuracy	0.703	0.7227
Train–dev gap	0.041	0.017

=> PCA improved dev accuracy

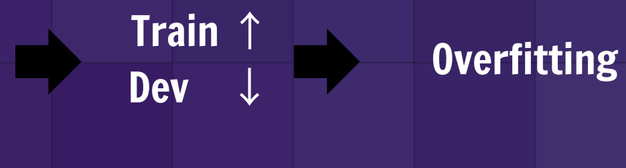
0.703 → 0.723

AI MISLED

1. Bigger model = better model

More neurons
did not consistently
improve performance

20	overfit	0.784	0.678
50	overfit	0.807	0.708
100	overfit	0.818	0.698



2. Higher train accuracy was misleading

Train ↑
Test ↓

3. Suggestions required validation

Some recommendations
looked promising
but generalized poorly

Final Reflection

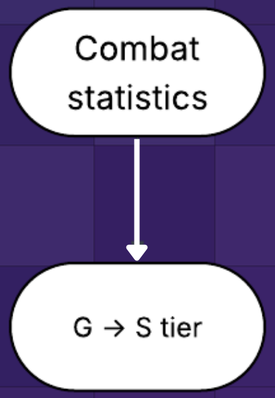
AI was useful for generating hypotheses, but experimental validation remained essential

=> AI supported decision-making,
but did not replace critical thinking

Conclusion

What We Built

A neural network classifier for predicting Pokémon strength tiers



Best Findings

✓ PCA improved development accuracy

✓ Standard model generalized more consistently

Best Dev Accuracy: 72.3%

Best Test Accuracy: 69.9%

What We Learned

Raw statistics alone cannot fully explain Pokémon strength

Feature engineering and contextual information matter significantly

Good machine learning is not only about accuracy, but also understanding what the model still does not understand

Future Improvements

1. Balance classes
   - SMOTE
   - class_weight

2. Better models
   - Random Forest
   - XGBoost
   - SVM

3. Richer features
   - Type
   - Ability
   - Legendary flag
   - Evolution stage

4. More evaluation
   - Cross validation

Thank you for listening